AI PROMPT HISTORY

TraceIQ

Supply Chain Traceability & AI Recall Command Center

Document purpose	Record of the prompts submitted to AI tools while developing this project.
Prompts recorded	11 entries (Prompts 1–11); full text reproduced unless noted
AI tools identified	Gemini 3.8 Flash (Google AI Studio); ChatGPT; Claude. Tool for Prompts 7–10 not recorded.
AI responses	Not reproduced (optional). Short outcome notes are given where the source transcript contained them.
Compiled	29 October 2026

1. Scope and completeness disclosure

Reasonable record, not a guaranteed complete export. The AI tools used do not provide a single complete export of every prompt. This document is compiled from the records that were available: the Gemini (Google AI Studio) transcript captured in the earlier prompt-history file, prompt text pasted into the request that produced this document, and the project files in the supplied zip. It may therefore omit short prompts, retries, follow-up messages or whole conversations that were not part of the supplied material.

Known limitations of this record

- Gemini (Google AI Studio) exposes the conversation as an on-screen transcript. It does not include per-prompt timestamps, so no send times are given for Prompts 1–4. The only date evidence is the attachment label “Paste October 28, 2026 - 7:48PM” and the project file dates (29 October 2026).
- The order of Prompts 7, 8, 9 and 10 relative to each other and to Prompts 1–6 is not established. They are numbered in the order they were supplied, not by confirmed chronology.
- The tool that received Prompts 7, 8, 9 and 10 is not stated in the supplied material. They are marked “not recorded” rather than guessed. Please fill these in before submission if you know the tool.
- Prompt 9 appears three times in the supplied material with identical text. It is listed once; the number of actual submissions is not known.
- Prompt 5 appeared next to a block of AI-written review feedback. That block is an AI response and is not reproduced. Prompt 6 repeats Prompt 4 and is summarised, not duplicated.
- The wording of Prompts 2 and 3 contains the original spelling and grammar mistakes. They have not been corrected.
- Text copied from chat interfaces lost some formatting (bold markers, bullet characters, line breaks inside lists). The wording is unchanged, but layout may differ slightly from the original.
- Prompts 7–10 describe features such as FastAPI, PostgreSQL, OpenRouter and OCR. These are requests made to AI tools, not evidence that those features were built. The supplied app in the zip is a React/Vite front-end project that uses local data and a Gemini-capable AI Studio configuration.

Suggested wording for the project report

> *“This AI prompt history was compiled from the records available to me. The tools used do not provide a complete prompt export, so it is a reasonable record rather than a guaranteed complete log of every prompt submitted.”*

2. AI tools used

Tool	Evidence in the supplied material	Prompts
Gemini 3.8 Flash (Google AI Studio)	Named in the transcript for each run. The project zip contains AI Studio files (metadata.json, .env.example comments, public/assets/aistudio/).	1, 2, 3, 4
ChatGPT (including Canvas)	The earlier history document records a ChatGPT conversation. Prompt 5 asks for the update “in Canvas”.	5, 6
Not recorded	Prompts 7–10 read like instructions for a coding assistant that can inspect a repository. The tool is not named in the supplied text.	7, 8, 9, 10
Claude (Anthropic)	This conversation, used to compile this document.	11

3. Prompt index

#	Tool	Prompt	Words
1	Gemini 3.8 Flash (Google AI Studio)	Master Prompt: Premium Enterprise UI Redesign	1733
2	Gemini 3.8 Flash (Google AI Studio)	“Make it more professional and easy to understand” (with pasted code)	17
3	Gemini 3.8 Flash (Google AI Studio)	“Change UI, it looks very dark”	15
4	Gemini 3.8 Flash (Google AI Studio)	UI Redesign Prompt — Clean, Bright & User-Friendly Supply Chain Dashboard	1740
5	ChatGPT (Canvas)	Review and improve the React application (traceability, reconciliation, recall workflow)	961
6	ChatGPT	UI Redesign Prompt (repeat of Prompt 4 with an added closing paragraph)	79
7	Not recorded in the supplied material (coding-assistant style prompt)	TraceIQ — Premium Enterprise Dashboard & UI/UX Implementation	1938
8	Not recorded in the supplied material (coding-assistant style prompt)	TraceIQ — Full-stack build plan (FastAPI + PostgreSQL + React)	1784
9	Not recorded in the supplied material (coding-assistant style prompt)	Master Engineering Prompt — Supply Chain Traceability, Document Intelligence & AI Recall Command Center	4597
10	Not recorded in the supplied material (coding-assistant style prompt)	Make “Trace a material or finished lot” work from uploaded evidence	797
11	Claude (Anthropic)	Request to compile this prompt-history PDF (and follow-up)	107

4. Full prompt text

Each entry shows the tool, the source of the text, and the prompt exactly as supplied. Square-bracketed notes in Prompts 6 and 11 are editorial notes added when compiling this record.

Prompt 1

Title	Master Prompt: Premium Enterprise UI Redesign
Tool	Gemini 3.8 Flash (Google AI Studio)
Source	AI_Prompt_History_Supply_Chain_Traceability.docx (source transcript)
Note	First prompt in the supplied Gemini history. Gemini run time shown in the transcript: 11m 11s; build reported successful.

PROMPT TEXT

Act as a senior UI/UX designer, enterprise SaaS product architect, and expert React frontend developer. Redesign my existing Supply Chain Traceability & AI Recall Command Center into a visually stunning, professional, modern enterprise application.

1. Main Objective

The current UI looks too simple, generic, and basic. Transform it into a premium-quality enterprise dashboard comparable to modern products such as Linear, Stripe Dashboard, Vercel, and high-end supply-chain analytics platforms.

The result should look like a real commercial software product suitable for a portfolio demonstration, technical presentation, and recruiter review.

Do not just change colors or add shadows. Redesign the entire visual experience, layout, component hierarchy, spacing, navigation, data visualization, and interactions.

2. Design Direction

Use a polished enterprise SaaS design system with:

Primary theme: Deep midnight navy, crisp white, cool slate, and sophisticated electric blue.

Accent colors: Teal for healthy states, amber for warnings, red for critical recall incidents, and violet for AI-related features.

Typography: Modern, clean typography such as Inter or Geist, with strong headings and readable data labels.

Visual style: Premium, minimal, data-rich, structured, elegant, and highly professional.

Layout: Spacious but information-dense, with consistent alignment, carefully designed cards, refined borders, subtle shadows, and balanced whitespace.

Interactions: Smooth transitions, subtle hover effects, active navigation indicators, polished modal dialogs, animated status transitions, and professional loading or empty states.

Icons: Use Lucide React icons consistently. Avoid emojis and random decorative graphics.

Avoid excessive gradients, oversized cards, too much empty space, unnecessary rounded boxes, and generic AI-generated dashboard styling.

3. Redesign the Application Shell

Create a professional application layout with:

Left sidebar

Elegant dark navy sidebar with a compact brand identity and custom supply-chain icon.

Navigation sections with clear labels and consistent icons.

Overview

Lot Genealogy

Recall Command Center

AI Investigation

Anomaly Detection

Audit & Reports

Active navigation should have a refined highlight, not just a basic background change.

Add subtle section separators and a bottom profile/settings area.

Make the sidebar collapsible on desktop and a drawer on mobile.

Top navigation

Breadcrumbs showing the current page.

Global search input with a search icon and keyboard shortcut hint.

A notification icon with a meaningful incident indicator.

A clear “DEMO ENVIRONMENT” label so users know this is synthetic data.

A small user profile menu.

Keep the header compact, elegant, and aligned with the main content.

Main content

Use a consistent content width and spacing system.

Add page titles, concise descriptions, optional action buttons, and contextual filters.

Ensure every page feels related to the same design system.

4. Dashboard — Make the First Impression Excellent

Redesign the Overview page as a premium operational command center.

Include:

A strong page heading: “Supply Chain Overview”.

A concise status summary showing overall network health and recent operational activity.

Four polished KPI cards:

Active Suppliers

Registered Raw-Material Lots

Finished-Goods Units On Hand

Open Quality Incidents

Each card should include a relevant icon, prominent metric, concise supporting information, and a subtle visual indicator where appropriate. Do not invent percentage changes or trends unsupported by the data.

Add these sections:

Supply Chain Inventory

A professional Recharts visualization showing inventory by warehouse.

Provide clear axes, readable tooltips, responsive sizing, and a useful legend if necessary.

Keep raw-material kilograms and finished-goods units separate to avoid misleading comparisons.

Incident Monitoring

Show critical quality issues, missing traceability links, and quantity mismatches.

Use clear severity labels and actionable navigation to the relevant feature.

Recent Activity

Present recent audit events as a clean, readable activity timeline.

Show action type, target entity, timestamp, and a concise event description.

Quick Actions

Trace a Lot

Investigate Anomalies

Simulate a Recall

Make the dashboard visually compelling on first load without overwhelming users.

5. Lot Genealogy — Build a Visual Traceability Experience

The existing traceability interface is too basic. Transform it into a professional supply-chain genealogy explorer.

Create a prominent search panel supporting lot IDs and production-order IDs.

Display the traceability path as a clear, horizontally organized visual flow:

Supplier → Raw Material → Production Order → Finished Goods → Warehouse / Shipment → Customer

Requirements:

Use distinct, visually coordinated node styles for suppliers, materials, production orders, finished goods, warehouses, shipments, and customers.

Show entity IDs, quantities, units of measure, status, and important relationships.

Use connector lines and directional arrows to make material flow easy to understand.

Visually distinguish upstream genealogy from downstream exposure.

Highlight the selected entity and its related path.

Make missing lots and broken links immediately noticeable using critical warning styling.

Provide compact entity details and contextual actions.

Make the flow horizontally scrollable when necessary and usable on smaller screens.

Include a clear summary of related production orders, inventory locations, shipments, and exposed customers.

Use the existing traceability engine and database relationships. Do not hardcode the rendered graph or fabricate relationships.

6. Recall Command Center — Design a Professional Incident Workflow

This is the most important feature. Make it feel like a real enterprise incident-response console.

Create a guided four-step workflow:

Identify Incident

Analyze Impact

Authorize Action

Reconciliation Report

Design a polished stepper with clear completion, current-step, and pending states.

Incident identification

Provide a refined lot or batch ID input.

Show supported example IDs.

Display invalid-target messages clearly.

Explain that the simulation uses local demonstration data.

Impact analysis

Display actionable finished-goods units, delivered units, affected customers, and estimated financial exposure.

Clearly explain the assumptions behind estimated financial exposure.

Separate on-hand inventory from in-transit and delivered shipments.

Show affected production orders and customer shipments in professional tables or compact panels.

Highlight missing upstream inputs and quantity mismatches.

Make the incident severity and potential business impact easy to understand.

Authorization

Show a clear quarantine action plan before execution.

Display exactly which lots, inventory records, and shipments will be affected.

Include a strong but professional confirmation modal.

Clearly distinguish destructive actions from ordinary navigation.

Prevent duplicate execution using the existing recall records.

Reconciliation

Show a professional outcome summary with affected lots, quarantined inventory records, held shipments, and shipments requiring customer follow-up.

Use readable reconciliation tables with status indicators.

Distinguish successful execution from pending follow-up tasks.

Do not claim that external notifications were sent when they were only flagged locally.

7. AI Investigation — Make It Look Like a Modern Investigation Workspace

Create a premium chat-based investigation interface.

Include:

A polished chat layout with clear user and assistant message alignment.

A refined input box with a send icon, keyboard support, and suggested prompts.

Attractive prompt suggestions such as:

“Trace RAW-1042 through the supply chain”

“Identify all traceability gaps”

“Explain the quantity mismatch”

“Draft a recall incident report”

Styled responses containing headings, summaries, warnings, and entity references where appropriate.

Clear loading feedback while a response is being generated.

A visible label that this is a simulated investigation assistant powered by local deterministic logic.

Keep the current rule-based responses unless actual AI integration is explicitly implemented. Do not pretend that an LLM or external AI service is connected.

8. Anomaly Detection — Create a Serious Quality-Control Workspace

Redesign the anomaly page with clear severity hierarchy and meaningful data presentation.

Include:

A top-level summary of critical issues, warnings, and verified checks.

Dedicated anomaly entries for missing raw-material lots, over-allocated quantities, and inventory-status mismatches.

Each entry should display its severity, affected entity, expected behavior, observed behavior, and current verification status.

Add contextual actions such as “Trace Entity” or “Open Recall Workflow”.

Make each issue dynamically reflect the current database state.

Clearly differentiate unresolved, verified, and resolved issues.

Avoid hardcoded success states that remain unchanged after data has been updated.

9. Audit Log & Reports — Professional Data Tables

Create a clean enterprise data-management interface.

Include:

Searchable and filterable audit events.

Filters for action type, target ID, and date where practical.

Clear timestamp formatting.

Consistent status and action badges.

Row hover states and readable table columns.

A polished JSON export button with a download icon.

An empty state when no events match the current filters.

Responsive table behavior for small screens.

Maintain the existing explanation that the audit ledger is a local demo and is not cryptographically immutable.

10. Component and Interaction Quality

Create reusable, consistent UI components for:

Buttons and icon buttons

KPI cards

Status badges

Search bars

Data tables

Modal dialogs

Empty states

Alert banners

Loading indicators

Page headers

Workflow steppers

Entity cards

Toast notifications

Use a consistent spacing, typography, border-radius, and color-token system.

Add:

Smooth transitions without distracting animations.

Clear focus states and keyboard accessibility.

Proper dialog semantics and Escape-key handling.

Helpful tooltips for unfamiliar supply-chain terms.

Clear validation errors and success/error feedback.

Responsive layouts for desktop, tablet, and mobile.

Consistent behavior for disabled and loading buttons.

11. Technical Requirements

My existing application is built using:

React functional components and hooks

Tailwind CSS

Lucide React

Recharts

React Context

Browser localStorage

Synthetic supply-chain entities and relationships

Please follow these rules:

Inspect and reuse the existing codebase before making changes.

Preserve the existing functionality, data models, seeded anomalies, traceability logic, recall workflow, reconciliation logic, and audit recording unless a specific bug requires a correction.

Do not remove features just to achieve a cleaner design.

Do not replace real computed values with hardcoded mock metrics.

Keep all navigation, search, charts, filters, buttons, modals, and workflow actions functional.

Keep the application compatible with its existing dependencies. Do not introduce unnecessary libraries.

Handle missing data, invalid IDs, empty results, and localStorage failures gracefully.

Do not claim that frontend-only functionality is secured, immutable, or connected to external systems.

Use responsive Tailwind utility classes and reusable components rather than duplicating large amounts of markup.

Preserve persistence across refreshes and ensure that the Reset Demo Data action continues to work.

Do not make unrelated business-logic changes during the visual redesign.

Verify that the finished application compiles and that every major page and critical workflow works.

12. Final Quality Standard

The final interface should feel like a polished commercial enterprise product, not a basic student dashboard.

Prioritize:

Strong visual hierarchy

High-quality spacing and typography

Visually meaningful data displays

Professional status indicators

Better traceability visualization

A premium recall workflow

Consistent styling across every page

Excellent responsive behavior

Functional interactions rather than decorative elements

Implementation instruction: Apply the redesign directly to the existing project. First inspect the current components and styles, then implement the full redesign across all six pages. Start with the shared application shell and design system, then redesign each page. Finish by testing the core workflows and fixing any build errors or broken interactions.

Do not return only a design explanation or sample screenshot. Deliver the actual updated, working React application while preserving the existing supply-chain functionality.

Prompt 2

Title	“Make it more professional and easy to understand” (with pasted code)
Tool	Gemini 3.8 Flash (Google AI Studio)
Source	AI_Prompt_History_Supply_Chain_Traceability.docx (source transcript)
Note	Submitted together with an attachment labelled “Paste October 28, 2026 - 7:48PM”, an earlier single-file React/JavaScript version of the app. The attachment is source code, not a prompt, and is included in the supplied project zip (public/assets/aistudio/). Wording and spelling are reproduced exactly as typed. Gemini run time: 9m 15s; build reported successful.

PROMPT TEXT

this is code which i was done previously so make it more professtional and easy understandable wedinterface

Prompt 3

Title	“Change UI, it looks very dark”
Tool	Gemini 3.8 Flash (Google AI Studio)
Source	AI_Prompt_History_Supply_Chain_Traceability.docx (source transcript)
Note	This run was cancelled and restored to the previous checkpoint, so it produced no changes. Wording and spelling are reproduced exactly as typed.

PROMPT TEXT

change ui its looks very dark make it simple interface like background white and other

Prompt 4

Title	UI Redesign Prompt — Clean, Bright & User-Friendly Supply Chain Dashboard
Tool	Gemini 3.8 Flash (Google AI Studio)
Source	AI_Prompt_History_Supply_Chain_Traceability.docx (source transcript)
Note	Gemini run time: 7m 19s; build reported successful (14 files edited).

PROMPT TEXT

Act as a senior UI/UX designer and React frontend developer. Redesign my existing Supply Chain Traceability & AI Recall Command Center with a clean, bright, modern, professional, and easy-to-understand interface.

1. Main Design Goal

The current website looks too dark, crowded, and visually complicated. I want a light-themed enterprise SaaS interface with clear sections, comfortable spacing, simple navigation, readable text, and well-organized information.

The website should feel modern, trustworthy, calm, intuitive, and professional — suitable for a real business application and a final-year project portfolio.

Do not merely change the background color. Improve the complete visual design, page layouts, spacing, typography, cards, tables, charts, navigation, and user interactions.

2. Use This Light Color Theme

Use a consistent color palette throughout the entire application.

Main background: Soft off-white #F8FAFC

Content surfaces: White #FFFFFF

Primary color: Professional blue #2563EB

Primary hover: Deep blue #1D4ED8

Main text: Dark slate #1E293B

Secondary text: Slate gray #64748B

Borders: Light gray #E2E8F0

Success: Soft green #16A34A

Warning: Warm amber #D97706

Critical alerts: Clear red #DC2626

AI feature accent: Soft violet #7C3AED

Use pale backgrounds for status indicators instead of large, saturated color blocks.

Examples:

Success: light green background with dark green text.

Warning: light amber background with dark amber text.

Critical: light red background with dark red text.

Informational: light blue background with dark blue text.

Avoid black or very dark backgrounds across large sections. Do not use a dark sidebar, dark dashboard panels, excessive gradients, or glowing effects. The entire application should consistently follow a light theme.

3. Create a Clean Application Layout

Sidebar

Replace the current dark sidebar with an elegant white or very light gray sidebar.

Include:

A small supply-chain logo and application name at the top.

Overview

Lot Genealogy

Recall Command Center

AI Investigation

Anomaly Detection

Audit & Reports

Design requirements:

Use simple, consistent Lucide icons.

Use dark slate text and muted gray icons for inactive navigation.

Highlight the active page with a pale blue background, blue text, and a thin blue indicator.

Keep navigation labels clear and easy to scan.

Add subtle separators instead of heavy borders.

Include a discreet profile or settings section at the bottom.

Allow the sidebar to collapse on desktop and become a navigation drawer on mobile.

Top Header

Create a clean white header with a subtle bottom border.

Include:

Current page title or breadcrumb.

A global search field.

A small notification icon.

A clearly visible but subtle “DEMO MODE” indicator.

A profile avatar and menu.

Keep the header compact and avoid unnecessary decoration.

Main Content

Use a soft off-white page background with white content cards.

Keep consistent left and right margins.

Use clear page titles, short descriptions, and appropriately placed action buttons.

Separate sections using whitespace and subtle borders.

Avoid placing every piece of information inside a separate card.

Prevent overlapping elements, unnecessary boxes, and crowded layouts.

Make the interface responsive for desktop, tablet, and mobile.

4. Dashboard — Simple, Readable and Informative

Redesign the Overview page to make key information understandable within a few seconds.

At the top, display:

Supply Chain Overview

Supporting text: “Monitor inventory, traceability, quality incidents, and recall activity.”

Create four compact KPI cards:

Active Suppliers

Registered Raw-Material Lots

Finished-Goods Units On Hand

Open Quality Incidents

Each card should have:

A small icon inside a subtle colored background.

A clear metric in large, readable text.

A short descriptive label.

A restrained visual style with a light border and minimal shadow.

Do not add fake percentage changes or invented performance statistics.

Below the KPI cards, create a well-organized two-column layout containing:

Inventory by Warehouse

Use Recharts with a clean, light background.

Use readable axis labels and subtle gridlines.

Include helpful tooltips.

Show separate quantities for raw materials measured in kilograms and finished goods measured in units.

Never combine incompatible units into one misleading chart.

Recent Activity

Display recent audit events in a simple timeline or compact list.

Include the action, target entity, timestamp, and description.

Use small status indicators instead of oversized colored panels.

Add a separate Quality Alerts section showing missing traceability inputs, quantity discrepancies, and unresolved inventory-status issues.

Include three quick actions:

Trace a Lot

Investigate Anomalies

Simulate a Recall

Make the dashboard clean, balanced, and easy to scan without excessive cards or decoration.

5. Lot Genealogy — Make Relationships Easy to Understand

Redesign the traceability page to clearly display the movement of materials throughout the supply chain.

Use this visual flow:

Supplier → Raw Material → Production Order → Finished Goods → Warehouse / Shipment → Customer

Requirements:

Use a light background and distinct but subtle entity styles.

Display entity IDs, quantities, units of measure, and statuses clearly.

Use clean connector lines and directional arrows.

Highlight the selected lot with a pale blue background and blue border.

Use a pale red alert style for missing lots and broken traceability relationships.

Clearly distinguish upstream traceability from downstream customer exposure.

Keep the graph horizontally scrollable when necessary.

Add a compact summary of affected production orders, finished goods, inventory locations, shipments, and customers.

Use readable tables or expandable detail panels where a complex graph would become crowded.

Keep this page functional. Generate relationships from the existing traceability engine rather than hardcoding visual connections.

6. Recall Command Center — Clear Four-Step Workflow

Redesign the recall page so that users can follow the process without confusion.

Use a clean four-step progress indicator:

Identify Incident

Analyze Impact

Authorize Action

Reconciliation Report

Use a thin progress line, numbered circles, simple labels, and clear completed/current/pending states.

Step 1: Identify Incident

Show a clearly labeled lot or batch input.

Include a helpful example such as RAW-1042.

Use concise instructions.

Show validation errors directly below the input.

Keep the primary button prominent but not oversized.

Step 2: Analyze Impact

Display four compact summary cards:

Actionable Units

Delivered Units

Affected Customers

Estimated Financial Exposure

Use clean tables for production orders and customer shipments.

Clearly distinguish:

Inventory currently on hand.

Shipments in transit.

Products already delivered.

Estimated financial exposure.

Use warning banners only when relevant. Explain assumptions behind estimates instead of presenting them as verified financial losses.

Step 3: Authorize Action

Display the exact affected lots, inventory records, and shipments.

Use a clear action summary with labels and counts.

Provide a professional confirmation modal.

Make critical actions visually distinct without covering the whole screen in red.

Prevent duplicate execution.

Step 4: Reconciliation Report

Show a simple outcome summary.

Display quarantine status, shipment holds, customer follow-up requirements, and quantity reconciliation.

Use a clean table with visible status badges.

Clearly distinguish completed actions from pending follow-up.

Do not claim that customer notifications were sent when the application only flags them for follow-up.

7. AI Investigation — Comfortable Chat Interface

Create a modern light-themed investigation assistant.

Requirements:

White chat surface with subtle borders.

Clear distinction between user messages and assistant responses.

Use pale blue for user messages and light gray for assistant messages.

Provide a comfortable input field with a clear send button.

Add suggestions such as:

Trace RAW-1042 through the supply chain

Find missing traceability links

Explain quantity discrepancies

Draft a recall incident report

Display responses with readable paragraphs, headings, and warning indicators.

Include subtle response-loading feedback.

Keep the simulated-AI disclaimer visible but unobtrusive.

Do not create an entirely new AI integration. Preserve the existing local rule-based investigation logic.

8. Anomaly Detection — Organized Quality Alerts

Present anomalies in a clear list or table rather than multiple oversized alert cards.

Each anomaly should show:

Severity

Anomaly title

Affected lot or production order

Expected behavior

Observed behavior

Current status

A contextual action

Use light severity indicators and concise text. Make critical issues easy to identify without making the whole page appear alarming.

Add functional actions such as:

Trace Entity

Review Recall Impact

Open Related Records

Make the anomaly status update dynamically based on the current database state.

9. Audit Log & Reports — Professional Tables

Use a white table surface with:

A clear page title.

Search input.

Compact filters.

Readable timestamps.

Light row separators.

Subtle row hover effects.

Consistent action badges.

A prominent but restrained Export JSON button.

Ensure long text does not break the layout. Add horizontal scrolling for small screens and a helpful empty state when no records match the filters.

Retain the disclaimer that the audit log is locally stored and is not a secure, immutable audit ledger.

10. Global Design Rules

Apply these rules consistently to every page:

Typography

Use Inter or an existing equivalent.

Page titles: approximately 24–28px.

Section headings: approximately 16–20px.

Body text: approximately 14–16px.

Supporting labels: approximately 12–13px.

Maintain comfortable line height and readable contrast.

Spacing

Use a consistent 8px-based spacing system.

Leave enough space between sections.

Keep card padding consistent.

Avoid both excessive whitespace and crowded information.

Cards

White background.

Thin #E2E8F0 border.

Small, subtle shadow only when useful.

Rounded corners around 10–12px.

No heavy gradients, glow effects, or thick decorative borders.

Buttons

Use blue for primary actions.

Use white with a gray border for secondary actions.

Use red only for genuinely dangerous actions.

Maintain consistent heights, padding, and hover states.

Charts and Tables

Favor clear labels over decorative styling.

Use light gridlines and subtle separators.

Ensure tables remain readable and responsive.

Preserve actual computed data.

Accessibility

Maintain strong text contrast.

Add visible keyboard-focus indicators.

Use meaningful button labels.

Support keyboard interaction and accessible dialog semantics.

Do not rely on color alone to communicate status.

11. Technical Requirements

My existing application uses React, Tailwind CSS, Lucide React, Recharts, React Context, and localStorage.

Modify the existing project directly.

Preserve all six pages and all working features.

Preserve the existing dataset, genealogy logic, recall workflow, duplicate-execution checks, reconciliation, and audit logging unless a targeted bug fix is necessary.

Do not replace real values with hardcoded mock metrics.

Do not remove functionality to simplify the design.

Do not introduce unnecessary libraries.

Create reusable components and consistent theme tokens.

Ensure buttons, filters, search, charts, modals, and navigation remain functional.

Keep local persistence and the Reset Demo Data feature working.

Handle missing data and browser storage errors gracefully.

Test the application for build errors, broken interactions, and responsive layout problems.

Final Instruction

Transform the entire existing application into a bright, clean, light-themed enterprise dashboard with a calm visual hierarchy and understandable information architecture.

The finished product must feel polished and easy to use, not dark, cluttered, overly colorful, or complicated.

Implement the actual React and Tailwind changes. Do not return only a design description or static mockup. Once finished, verify all six pages, especially the traceability graph and the complete recall workflow.

Prompt 5

Title	Review and improve the React application (traceability, reconciliation, recall workflow)
Tool	ChatGPT (Canvas)
Source	Pasted text supplied with the request; tool inferred from the earlier history document and the “Canvas” instruction
Note	This text appeared in the supplied material directly after a block of AI-written review feedback (an AI response, not a prompt; not reproduced here because responses are optional). The tool is inferred from the “Update the actual application in Canvas” instruction and from the earlier history document, which records this prompt as submitted in ChatGPT.

PROMPT TEXT

Review and improve my existing React application, “Supply Chain Traceability & AI Recall Command Center.”

I have an existing implementation. Do not rebuild it from scratch. Preserve the working features, fix the bugs, simplify the code, and make the interface clean, professional, and suitable for a project demonstration.

1. Professional UI

Create a polished enterprise SaaS interface with:

- A compact dark navy sidebar.
- A clean white and light-gray workspace.
- Professional typography, consistent spacing and subtle borders.
- Restrained blue accents, green success indicators, amber warnings and red critical alerts.
- Responsive layouts, clean tables, KPI cards and readable charts.
- Consistent button sizes, status badges, page headings and empty states.

Remove unnecessary visual clutter, excessive borders, redundant headings and oversized elements. Prioritize clarity and usability over decoration.

2. Simplify the code

Keep the implementation compact and modular. Use the existing React, TypeScript if available, Tailwind CSS, Lucide icons and Recharts dependencies.

Organize the code into logical responsibilities:

- Application layout and navigation.
- Shared UI components.
- Seeded demo data and types.
- Traceability and quantity calculations.

- Recall workflow and audit history.
- Dashboard and feature screens.
- AI investigation logic.

Avoid creating a separate file for every small component. Keep related functions together when practical, but separate business logic from UI rendering.

Fix React state issues caused by defining stateful page components inside the main App component. Ensure that switching pages does not unexpectedly reset form inputs, chat history or workflow state.

3. Fix the traceability engine

The current traceLot function has incomplete upstream tracing and does not correctly handle production order searches.

Implement reliable upstream and downstream traversal:

- Raw material → production orders → finished lots → warehouse stock → shipments → customers.
- Finished lot → its production order → consumed raw materials and suppliers.
- Production order ID → its input materials and output lot.

Prevent infinite loops using visited sets. Detect missing lot references and report traceability gaps.

Calculate the impact from the actual relationships in the dataset. Do not hardcode affected customers or finished lots.

4. Fix quantity reconciliation

The existing code adds inventory and shipment quantities together without sufficiently distinguishing their statuses.

Separate:

- Produced quantity.
- Current available stock.
- Quarantined stock.
- In-transit quantity.
- Delivered quantity.
- Returned quantity.
- Destroyed quantity.
- Unexplained quantity differences.

Track units of measure consistently. Never add kilograms of raw materials directly to units of finished products.

Prevent double-counting when a shipment moves from in-transit to delivered. Validate quantities against production output and recorded inventory movements.

Display the calculation and reconciliation status clearly.

5. Improve the recall workflow

Keep the recall workflow simple and guided:

1. Identify the suspect lot.
2. Simulate the impact.
3. Review affected lots, inventory and customers.
4. Confirm the authorized action.
5. Quarantine affected stock and place eligible shipments on hold.
6. Review reconciliation and outstanding actions.

Simulation must never mutate operational demo data.

Execution must require explicit confirmation and record audit events. Quarantined inventory must no longer count as available stock.

Do not claim that external shipments were stopped or real customers were notified. Clearly identify these as simulated actions.

Prevent recall closure when mandatory actions remain incomplete or quantities are unexplained.

6. Improve the dashboard

Use a small set of meaningful KPI cards:

- Total suppliers.
- Raw and finished lots.
- Available inventory.
- Open quality incidents.
- Active recalls.
- Affected customers.
- Unreconciled exceptions.

Include a warehouse inventory chart, recent audit activity, high-risk lots and a clear shortcut to simulate a recall.

All figures must be calculated from the shared dataset and update after relevant actions.

7. Complete all navigation pages

Make every existing sidebar item functional:

- Overview.
- Lot Genealogy.
- Recall Command Center.
- Quality Control.
- Inventory & Stock.
- AI Investigation.
- Anomaly Detection.
- Audit & Reports.

Implement basic useful tables and filters where appropriate. Do not display generic “feature in development” pages for these sections.

8. Make the AI assistant honest and useful

Keep the AI assistant simulated for now; do not require API keys.

Generate responses from the current demo data and traceability calculations. Remove hardcoded customer names, fixed impact summaries and claims that records were securely accessed unless the application actually supports those claims.

Support questions about:

- Affected products and customers.
- Inventory exposure.
- Traceability gaps.
- Quantity discrepancies.
- Recall report drafting.

Clearly label the assistant as “Demo AI — simulated responses.” State when the available data is insufficient.

9. Fix anomaly detection and exports

Implement four distinct test scenarios:

1. A finished lot references a missing input lot.

2. Delivered quantity exceeds valid produced or available quantity.

3. A deliberately faulty test-only trace simulation misses a downstream customer, while the correct production trace engine identifies that customer.

4. Recalled inventory incorrectly remains available for sale.

Show expected behavior, detected condition, severity and recommended action.

Make JSON and CSV exports functional. Ensure that the audit history records trace simulations, recall actions, quarantine changes and reset events.

10. Final quality checks

Before finishing:

- Fix syntax errors, missing imports and React warnings.

- Fix broken buttons, navigation and empty states.
- Check calculations against the seeded data.
- Ensure localStorage updates correctly and reset restores the original dataset.
- Test the complete RAW-1042 recall scenario from simulation through quarantine and reconciliation.
- Preserve the existing useful features.
- Avoid adding unnecessary dependencies or backend infrastructure.

Final deliverable

Update the actual application in Canvas. Keep the code simple, professional, readable and easy to extend later into a FastAPI and PostgreSQL backend.

Do not return only recommendations or isolated code snippets. Implement the changes in the existing project, verify the important workflows, and summarize what was fixed and what remains simulated.

Prompt 6

Title	UI Redesign Prompt (repeat of Prompt 4 with an added closing paragraph)
Tool	ChatGPT
Source	Pasted text supplied with the request; earlier history document records it as part of the ChatGPT conversation
Note	Not reproduced a second time. The visible text matches Prompt 4 except for formatting artifacts from copying and one added closing paragraph, which is shown below.

PROMPT TEXT

[The submission repeated the full text of Prompt 4 (“UI Redesign Prompt — Clean, Bright & User-Friendly Supply Chain Dashboard”). Apart from formatting artifacts from copying, the visible text matches Prompt 4 and is not reproduced a second time. The only addition was this closing paragraph:]

This is a good portfolio prototype for a B.Tech Computer Science project. Be transparent in your project documentation that the AI, authorization, notifications, and audit ledger are currently simulated rather than production integrations.

Prompt 7

Title	TraceIQ — Premium Enterprise Dashboard & UI/UX Implementation
Tool	Not recorded in the supplied material (coding-assistant style prompt)
Source	Pasted text supplied with the request
Note	The supplied text does not say which tool received this prompt, when it was sent, or in what order relative to the other coding-assistant prompts (8–10). Confirm the tool before submission.

PROMPT TEXT

TRACEIQ — PREMIUM ENTERPRISE DASHBOARD & UI/UX IMPLEMENTATION

Role

Act as a principal product designer, senior React/TypeScript engineer, and enterprise SaaS dashboard specialist.

You are working on the existing TraceIQ Supply Chain Traceability, Quality & AI Recall Command Center.

Your primary goal in this phase is to create a visually polished, data-rich, highly usable application interface. Focus on frontend layout, dashboard hierarchy, charts, KPIs, navigation, interactions, responsive behavior, and consistent design.

Do not rebuild the entire application or replace the current visual identity. Inspect the existing frontend and improve it incrementally.

1. First Page: Executive Dashboard

After successful login, users must land on the Dashboard by default.

The Dashboard should provide an executive overview of the complete supply-chain operation, document intelligence, quality validation, inventory risk and recall activity.

Design it as a professional enterprise SaaS application inspired by modern operations command centers. Retain the current dark teal theme and functional light/dark toggle.

Avoid a generic template with a few cards and a large empty chart.

Dashboard layout

A. Application shell

Create a persistent left sidebar with:

- TraceIQ logo and application name
- Overview
- Supply Chain
- Suppliers
- Materials & Lots
- Production
- Inventory & Warehouses
- Shipments & Deliveries
- Documents
- Validation Center
- Traceability Explorer
- Recall Management
- Corrective Actions
- Reports
- Audit Trail
- Settings

Use appropriate icons, compact spacing, clear active states, and collapsible navigation.

At the top, provide:

- Breadcrumbs or page title
- Global search
- Date-range selector
- Notifications
- Light/dark theme toggle
- User profile menu

Use a clean top bar without overcrowding it.

B. Dashboard heading

Display:

Supply Chain Overview

Subtitle:

“Monitor operational health, document compliance, inventory risk and recall readiness.”

Include:

- Date range: Today, 7 days, 30 days, 90 days
- Refresh action
- Optional facility selector if facilities exist in the data model
- Last-updated timestamp

The selected date range should affect charts and relevant metrics when backend data supports it.

C. KPI cards

Create a responsive grid of six KPI cards.

1. Suppliers

- Total suppliers
- Active suppliers
- Optional supplier compliance percentage

2. Material Lots

- Total tracked lots
- Lots awaiting inspection
- Lots on hold or quarantined

3. Production Orders

- Active orders
- Completed orders
- Orders with exceptions

4. Inventory

- Eligible available stock
- Quarantined or blocked lots
- Stock-risk indicator

5. Documents

- Total documents processed
- Pending review
- Failed or incomplete extractions

6. Quality & Validation

- Checks passed
- Critical failures
- Open findings

Every KPI card should include:

- A subtle icon
- Clear label
- Large primary value
- Small supporting metric
- Optional trend versus the previous equivalent period, only when calculable
- A clickable interaction that opens the relevant filtered screen

Use consistent card heights, restrained borders and shadows, legible typography, and generous whitespace.

Never invent percentage changes or trends when data is unavailable.

D. Main chart: Area chart

Make the central chart a prominent, wide Supply Chain Health Over Time area chart.

It should be the primary visual element on the dashboard.

Use a smooth, responsive area chart with a subtle gradient fill, clear axes, readable tooltips, a date-range selector, and an accessible legend.

Provide selectable series such as:

- Documents processed
- Validation checks passed
- Open validation findings
- Recall or quality exceptions

Choose sensible defaults that make operational activity easy to understand.

Requirements:

- Use Recharts or the existing charting library.
- Display multiple series only when the values have meaningful and compatible units.
- Prefer separate selectable series rather than placing unrelated quantities on one misleading scale.
- If counts are compared, label the axis clearly.
- If the chart shows rates, label it as a percentage and use a percentage axis.
- Use the backend’s time-series data when available.
- Provide loading, empty and error states.
- Add hover tooltips with dates, series names and actual values.
- Keep the chart responsive to the available content width.
- Do not use a static screenshot in place of a real chart.

Add small summary labels above the chart, such as total processed, current open findings or latest activity, based on actual data.

E. Operational status and risk panels

Under the main chart, create a two-column section.

Left: Operational Activity

- Recent production updates
- Incoming goods receipts
- Documents uploaded
- Validation runs completed
- Shipments dispatched
- Recall workflow changes

Each activity row should show an icon, short description, linked entity, timestamp and status.

Right: Risk & Exceptions

- Critical validation failures
- Lots awaiting quality disposition
- Blocked or quarantined inventory
- Missing lot genealogy
- Documents requiring review
- Active recalls requiring attention

Use restrained severity indicators. Critical issues should be prominent without making the entire interface visually alarming.

Clicking an item must open the relevant detail screen or a filtered list.

F. Additional dashboard panels

Add a final responsive row for:

Document Processing

- Queued
- Processing
- Review required
- Failed

Recall Readiness

- Active recalls
- Customers awaiting acknowledgement
- Outstanding recovery quantities
- Unresolved closure blockers

Recent Validation Findings

- Rule ID
- Finding description
- Severity
- Entity
- Status
- View details action

Avoid duplicating all KPI information unnecessarily. These panels should provide operational detail, not repeat the same numbers.

2. Visual Design System

Retain the existing dark teal identity and make it consistent across all screens.

Dark theme

- Deep teal or blue-green application background
- Slightly elevated sidebar and top navigation
- Distinct but subtle card surfaces
- High-contrast primary text
- Muted secondary text
- Teal accent for primary actions and selected navigation
- Restrained red, amber and green semantic indicators

Light theme

- Warm or neutral off-white page background
- White or lightly tinted cards
- Dark, readable text
- Subtle borders
- Teal accent colors
- Consistent semantic status colors

Design rules

- Use design tokens for colors, spacing, typography, borders, shadows and radii.
- Avoid excessive gradients, glassmorphism, neon effects and oversized decorative elements.
- Use a consistent 8-point spacing rhythm where practical.
- Make headings, labels and values visually distinct.
- Use tabular numerals for KPI values.
- Provide skeleton loading states.
- Use accessible color contrast.
- Include clear hover, focus, selected and disabled states.
- Respect reduced-motion preferences.
- Ensure both themes work across every page, not only the header.

The result should feel like a real enterprise product used daily by supply-chain managers, quality teams, logistics analysts and recall coordinators.

3. Improve Existing Screens

Preserve and refine the existing:

- DocumentsScreen.tsx
- ValidationScreen.tsx
- Navigation and routes
- Typed API services
- Existing UI components

Do not delete working document upload, extraction review, evidence links, activity history or validation features.

Make all screens share the same design system, typography, page width, buttons, tables, filters, dialogs and status badges.

Documents Center

Preserve the three-panel document intelligence layout:

- File preview
- Editable extraction review
- Validation results and evidence/activity context

Improve responsive behavior so panels stack or become switchable tabs on smaller screens.

Provide clear upload actions, processing states, field-level validation, document metadata and error feedback.

Validation Center

Preserve:

- Validation findings
- Corrective actions
- Check-run history

Add compact summary metrics, search and filters, a clear severity hierarchy, and a detail view showing expected versus actual values and evidence.

Other screens

Apply the same shared layout and design tokens to existing supply-chain, traceability and recall screens.

Do not create empty placeholder pages that look finished but have no functionality. Clearly label incomplete modules.

4. Component Architecture

Reuse shared components instead of copying markup between pages.

Create or extend components for:

- ApplicationShell
- SidebarNavigation
- HeaderBar
- PageHeader
- KpiCard
- AreaTrendChart
- StatusBadge
- ActivityFeed
- RiskExceptionList
- DataTable
- FilterBar
- EmptyState
- ErrorState
- LoadingSkeleton
- ConfirmationDialog
- DocumentStatusCard

Keep the existing repository structure wherever practical. Do not create unnecessary folders or split every small component into a separate file.

Use TypeScript interfaces for all API response and component data structures.

5. Dashboard Data and API Integration

The dashboard must use real backend data whenever the API supports it.

Inspect existing endpoints before adding new ones.

If required, add a dashboard endpoint such as:

GET /api/v1/dashboard/summary?period=30d

It may return:

- KPI totals
- KPI supporting metrics
- Time-series data
- Operational activity
- Risk and exception summaries
- Document processing states
- Recall readiness summaries
- Recent validation findings
- Last-updated timestamp

Add a dedicated typed frontend dashboard API service only if it fits the existing API architecture.

Avoid making many unnecessary requests when a summary endpoint can efficiently provide the dashboard data.

Use appropriate caching and refresh behavior.

If an endpoint or metric does not exist:

- Implement the required backend aggregation when practical.
- Otherwise show a clear unavailable state or use explicitly labelled demo data in development mode.
- Never invent live metrics or fabricated trends.

Ensure the dashboard remains useful when the database is empty.

6. Chart Data and Interactions

Use a typed time-series response, for example:

- Date
- Documents processed
- Checks passed
- Open findings
- Quality exceptions

The API must return values with a clearly defined aggregation period and consistent meaning.

Implement:

- Tooltip details
- Series toggles
- Date-range changes
- Loading and error states
- Empty-state handling
- Responsive layout
- Accessible chart descriptions

Do not combine percentages and raw counts on the same axis without an explicit, understandable design.

Keep chart colors consistent with the current theme and status conventions.

7. Frontend Engineering

Use the existing React, TypeScript and Tailwind setup.

Requirements:

- Strict TypeScript
- Reusable components
- No unnecessary any types

- No duplicated API logic
- No fake buttons
- No dead navigation links
- No console errors
- No broken imports
- No regressions to document or validation workflows

Use the project’s existing charting and component libraries where possible. Install dependencies only when necessary.

Do not introduce a new state-management library if the existing approach is sufficient.

8. Responsive Behavior

Desktop:

- Persistent collapsible sidebar
- Wide area chart
- Six KPI cards arranged efficiently
- Two-column operational sections

Tablet:

- Collapsible sidebar
- KPI cards in two or three columns
- Chart remains readable
- Operational panels stack when necessary

Mobile:

- Compact navigation drawer
- KPI cards in one or two columns
- Chart retains readable tooltips and labels
- Tables support horizontal scrolling or responsive row layouts
- Document panels become tabs or stacked sections
- Touch targets remain accessible

9. Validation and Acceptance Criteria

After implementation:

1. The Dashboard is the first page after login.
2. The dashboard displays six KPI cards.
3. A real interactive area chart appears prominently.
4. Chart and KPI values use backend data or clearly labelled demo data.
5. Date-range changes work.
6. Operational activity and risk panels are clickable.
7. Dark and light themes work across all screens.
8. Existing document and validation workflows still function.
9. Empty, loading, error and populated states are implemented.
10. Navigation and responsive layouts work at desktop, tablet and mobile widths.
11. No critical browser-console errors occur.
12. Existing lint and build commands pass.
13. Browser tests cover the dashboard and key interactions.
14. Backend tests are run if dashboard APIs or aggregations change.

Start by inspecting the existing codebase. Implement the UI improvements directly in the existing project, then run the checks and report the exact files changed and results.

Priority for this phase: frontend UI/UX first. Make the dashboard and shared application shell visually complete and functional before spending time on unrelated backend features.

Prompt 8

Title	TraceIQ — Full-stack build plan (FastAPI + PostgreSQL + React)
Tool	Not recorded in the supplied material (coding-assistant style prompt)
Source	Pasted text supplied with the request
Note	The supplied text does not say which tool received this prompt, when it was sent, or in what order relative to the other coding-assistant prompts (7, 9, 10). Confirm the tool before submission.

PROMPT TEXT

You are working on TraceIQ: Supply Chain Traceability & AI Recall Command Center.

PROJECT LOCATION

Inspect the existing workspace first. It currently contains traceiq-frontend/ and traceiq-backend/. Continue in those directories if present. Do not create duplicate supply-chain-frontend/ or supply-chain-backend/ folders, rename the project, or discard existing work without a clear reason.

GOAL

Build a working end-to-end supply-chain traceability and recall application. A user must be able to select a potentially defective raw-material lot, trace affected production and finished lots, inventory, shipments, and customers, review deterministic validation findings, simulate and approve a recall, track containment and recovery, reconcile quantities, and produce an auditable closure report.

FIRST: INSPECT AND PLAN

Before editing:

1. Read the existing frontend and backend files, package configuration, data, routes, and current UI.

2. Review the supplied project brief and diagrams as product/workflow context.

3. Identify existing features and code worth retaining, including duplicate screens, data models, contexts, API calls, and any current defects.

4. Create IMPLEMENTATION_PLAN.md at the workspace root. It should describe the current state, target structure, key data/API contracts, implementation phases, and decisions that need validation.

5. Then execute the plan in phases. Do not stop after writing the plan. Keep the existing useful UI and behavior while moving toward this architecture.

ARCHITECTURE RULES

- Keep the frontend and backend separate.
- The FastAPI backend and PostgreSQL database own authoritative data, calculations, quality rules, traceability, and recall state.
- The frontend presents data and collects user actions. It must not be the authority for recall scope, quality outcomes, or traceability calculations.
- The AI may summarize verified records, explain findings, identify evidence gaps, and draft reports. It must not invent affected entities, change validated recall scope, bypass a failed gate, or approve or execute a recall.
- Require explicit human approval for recall activation and record the approver and decision in the audit history.
- Record important mutations and workflow transitions in an append-only audit history.
- Use deterministic Python rules for initial quality, anomaly, quantity, and recall calculations. Do not add a graph database, machine-learning pipeline, Redis, or background workers to the first milestone.
- Keep the codebase compact. Use the requested files and organize them with clearly named classes/functions and sections. Split a file when its size makes it hard to maintain; do not create a large number of tiny modules prematurely.

- Do not fake API success or present demo data as live backend data. If demo mode is needed, make it explicit and keep its fixtures separate from API services.
- Never put database credentials or OpenRouter keys in frontend code.

FRONTEND STRUCTURE

Use the existing traceiq-frontend/ project and organize src around these responsibilities:

src/
app/
App.tsx
routes.ts
providers.tsx
components/
layout/ # App shell, sidebar, header
common/ # Shared badges, cards, tables, dialogs, toasts, loading/error/empty states
features/
dashboard/
suppliers/
traceability/
quality/
inventory/
recalls/
ai-investigation/
anomalies/
audit/
reports/
screens/ # One route-level screen per product area
hooks/ # Screen data loading and user-action state
services/
api/
client.ts
dashboardApi.ts
suppliersApi.ts
lotsApi.ts
traceabilityApi.ts
inventoryApi.ts
qualityApi.ts
recallsApi.ts
anomaliesApi.ts
auditApi.ts
reportsApi.ts
aiApi.ts
types/ # Shared domain and API request/response types
data/demo/ # Optional, clearly separated demo fixtures
utils/
styles/

Frontend expectations:

- Replace the current activeTab-only page switching with URL-based routes and working browser navigation.

- Keep screens focused on composing components and connecting hooks.
- Put all HTTP calls in services/api. Do not fetch from screens or import backend business logic into the frontend.
- Consolidate duplicate page/screen implementations, contexts, and components carefully. Migrate useful behavior before deleting old files.
- Replace competing supply-chain data types with one consistent TypeScript contract aligned with FastAPI's Pydantic request/response schemas.
- Show loading, empty, validation, and recoverable error states for API-backed screens.
- Keep the existing visual design where useful. Make the layout responsive for desktop and tablet use.
- Provide an interactive upstream/downstream genealogy visualization using React Flow or a compatible graph library. Use the existing charting library where it already fits.
- Configure the frontend API base URL through an environment variable. Add a development proxy if useful so the app can reach FastAPI at /api/v1.

BACKEND STRUCTURE

Use Python, FastAPI, Pydantic, SQLAlchemy, and PostgreSQL. Start with this compact layout:

traceiq-backend/

app/

main.py # FastAPI app, middleware, controller/router registration

config.py # Environment settings

database.py # PostgreSQL connection and session management

controllers.py # API routes and HTTP handling, grouped by domain

services.py # Business rules and workflow orchestration

models.py # SQLAlchemy entities and Pydantic schemas in clearly separated sections

ai.py # OpenRouter client and evidence-grounded AI workflows

tests/

test_app.py

.env.example

Dockerfile

requirements.txt

README.md

Keep controllers thin: validate/parse HTTP input, call services, and return typed responses. Put deterministic business logic in services.py. Use database transactions for operations that must not partially apply, especially recall activation, quarantine, shipment holds, and reconciliation updates. Add files only when the compact layout becomes difficult to maintain.

CORE PRODUCT WORKFLOWS

1. Supply-chain lifecycle

- Supplier registration and qualification.
- Raw-material receipt with supplier, lot identity, certificates, received quantity, and inspection evidence.
- Quality inspection and disposition: approve, quarantine, or open a supplier corrective action.
- Production order and bill-of-material relationships recording consumed raw lots and output finished lots.
- Finished-product inspection and disposition.
- Warehouse inventory and movements, allocations, shipments, delivery confirmation, and returns.

2. Quality gates and corrective actions

Implement versioned deterministic checks with results PASS, FAIL, PENDING, or INCOMPLETE.

- A business event selects the applicable rule version and authorized source records.
- Record the check result and supporting evidence.
- FAIL or INCOMPLETE opens a corrective action.

- Allow data correction or issue resolution and record a new check run.
- Permit the next workflow transition only when all required gates pass.
- Preserve earlier runs and evidence for auditability.

3. Traceability and quantity reconciliation

- Trace upstream from finished lot to production order, consumed input lots, and suppliers.
- Trace downstream from raw material through production, finished lots, stock, shipments, and customers.
- Represent missing links explicitly; never silently omit them.
- Reconcile produced, consumed, on-hand, quarantined, shipped, delivered, returned, destroyed, and documented process-loss quantities with units.
- Flag unexplained differences, over-allocation, duplicate/missing links, and stock that remains sellable after recall containment.
- Make rules deterministic and return findings with evidence and affected entity identifiers.

4. Incident and recall workflow

- Open an investigation from a failed inspection, supplier alert, complaint, or manually reported issue.
- Calculate recall impact from authoritative records and show scope, quantities, stock locations, shipments, customers, uncertainty, and data gaps.
- Support simulation and review before activation.
- Require an authorized human approval.
- On activation, transactionally quarantine affected available stock and place holds on eligible affected shipments.
- Track customer notification status, acknowledgements, returns, recovery, destruction, and other dispositions. For the prototype, record notification events without sending real emails.
- Reconcile all affected quantities and exceptions.
- Allow closure only when configured mandatory actions pass; otherwise keep the recall open and explain the blockers.
- Produce a closure report linked to the supporting evidence and audit events.

5. AI investigation

- Call OpenRouter from the backend only.
- Send only the minimum authorized records and deterministic findings needed for the request.
- Require structured responses validated against Pydantic schemas.
- Return evidence references, uncertainties, missing evidence, and suggested next steps.
- Clearly label AI-generated summaries and drafts.
- If no AI key is configured, report that AI is unavailable or use a clearly labeled local demo response; never imply a real model call occurred.

API CONTRACT

Version endpoints under /api/v1. Provide typed, documented endpoints covering:

- Dashboard summary and operational metrics.
- Supplier and product records.
- Lots, production orders, inventory, shipments, and customer records.
- Quality checks, check runs, findings, and corrective actions.
- Upstream/downstream genealogy and quantity validation.
- Recall impact simulation, approval, activation, actions, reconciliation, and closure.
- Anomalies, audit events, reports, and AI investigations.

Use pagination/filtering for potentially large lists. Return consistent error responses. Align frontend TypeScript types with Pydantic schemas. Document the API in FastAPI OpenAPI output and the project README.

DATASET AND REQUIRED ANOMALY SCENARIOS

Create a deterministic synthetic dataset generator with seed 20261022. Target approximately:

- 500 suppliers
- 10,000 raw-material lots

- 5,000 production orders
- 8,000 finished lots
- 20,000 deliveries

Keep planted test ground truth separate from operational records. Include these known scenarios and verify the deterministic engines detect them:

1. A finished lot references a missing input lot.
2. Shipped or delivered quantity exceeds produced quantity.
3. A recall trace misses a downstream customer.
4. Recalled inventory remains available for sale.

Use PostgreSQL constraints and transaction-safe writes where they protect integrity. Avoid loading the entire dataset into the browser.

IMPLEMENTATION PHASES

Phase 1 — Foundation

- Inspect and document the current app.
- Consolidate the frontend domain/API types.
- Set up FastAPI, PostgreSQL configuration, environment examples, and app startup.
- Establish frontend routes, app shell, and API client.

Phase 2 — Supply-chain records

- Implement suppliers, lots, production, inventory, shipments, and customers.
- Add deterministic seed data/generator and basic frontend list/detail workflows.

Phase 3 — Quality and traceability

- Add quality gates, versioned check runs, corrective actions, genealogy, quantity validation, and anomaly findings.
- Connect the traceability screen to backend results.

Phase 4 — Recall command center

- Implement impact simulation, human approval, transactional containment, customer/action tracking, reconciliation, and closure reporting.
- Complete the end-to-end defective-lot workflow.

Phase 5 — AI and reporting

- Add the server-side OpenRouter integration with evidence-grounded structured results.
- Add investigation summaries, report drafts, and audit-linked exports.

Phase 6 — Hardening

- Document local setup and API configuration.
- Add clear operational errors and responsive states.
- Review database transactions, audit coverage, access boundaries, and performance for the supplied dataset size.

VALIDATION

- Add backend pytest coverage for API behavior, rule outcomes, quantity reconciliation, transaction safety, audit history, and the four planted anomaly scenarios.
- Add a small Playwright end-to-end workflow for investigating and simulating a recall, with human approval required before activation.
- Mock OpenRouter in tests; tests must not require a real API key or make real external calls.
- Run the relevant checks after implementation and report exactly which commands passed or failed.

MILESTONE ACCEPTANCE

A user can identify a defective raw lot such as RAW-1042, view its complete upstream/downstream trace, see missing links and quantity findings, review AI assistance grounded in those findings, approve the recall as a human, contain affected stock and shipments, record recovery actions, reconcile quantities, and generate an auditable closure report. Every result must come from backend records and deterministic rules; AI must not

determine or execute the recall.

At the end, summarize the structure and workflows implemented, list the main API routes, state the setup commands, and report validation results and remaining limitations.

Prompt 9

Title	Master Engineering Prompt — Supply Chain Traceability, Document Intelligence & AI Recall Command Center
Tool	Not recorded in the supplied material (coding-assistant style prompt)
Source	Pasted text supplied with the request
Note	The identical text appears three times in the supplied material. It is reproduced once here. The material does not show whether these were three separate submissions or repeated pasting, so the number of times it was submitted is not established. The tool is not recorded; confirm before submission.

PROMPT TEXT

MASTER ENGINEERING PROMPT — SUPPLY CHAIN TRACEABILITY, DOCUMENT INTELLIGENCE & AI RECALL COMMAND CENTER

1. Your Role

Act as a principal software architect, senior Python backend engineer, senior React/TypeScript frontend engineer, database engineer, LLM integration engineer, document AI/OCR specialist, QA engineer, and application security engineer.

Build a complete, integrated, enterprise-grade web application named:

Supply Chain Traceability, Quality & AI Recall Command Center

Do not deliver merely a prototype, landing page, static dashboard, architecture document, pseudocode, or a few disconnected API endpoints.

Implement the actual frontend, backend, database schema, document processing pipeline, LLM integration, validation engine, supply-chain genealogy, recall workflow, authentication and authorization foundations, automated tests, seeded demo data, Docker configuration, and setup documentation.

The application must be runnable locally, with clear instructions for configuring and running every component.

Prioritize correctness, modularity, traceability, explainability, security, testability, and a professional user experience.

2. Primary Technology Stack

Frontend

- React
- TypeScript with strict mode
- Vite
- Tailwind CSS
- shadcn/ui or an equivalent accessible component library
- React Router
- TanStack Query for API state
- React Hook Form and Zod for form validation
- React Flow for interactive supply-chain genealogy
- Recharts or Plotly for analytics
- A reliable PDF viewer and image preview
- Lucide icons
- Responsive desktop, tablet, and mobile layouts

Backend

- Python 3.12+
- FastAPI
- Pydantic v2
- SQLAlchemy 2.x
- Alembic migrations
- PostgreSQL
- pytest
- HTTPX for API integration tests
- Structured application logging

Background Processing and File Storage

- Redis and Celery, or a clearly implemented equivalent, for asynchronous document processing and large imports
- Local filesystem or MinIO for development file storage
- S3-compatible private object storage adapter for deployment
- SHA-256 checksums for uploaded files
- PyMuPDF for PDF text extraction
- Tesseract or PaddleOCR for scanned documents
- Support for DOCX text and table extraction
- Safe MIME detection and file validation

LLM Integration

- OpenRouter API
- A configurable model registry and task-based model routing
- An internal LLM gateway that supports timeouts, retries, token limits, usage tracking, structured outputs, and model fallbacks where compatible
- Environment-based configuration for the OpenRouter API key and model preferences

Never expose API keys or other secrets to the frontend.

Do not hardcode one model throughout the application. Make models configurable by task and allow a user with appropriate permissions to select from administrator-approved models.

3. Architecture Requirements

Build a modular monolith, not a collection of premature microservices.

Keep the number of primary source files manageable. Use approximately 4–5 main folders per application layer and a small number of substantial modules. Split a module only when doing so materially improves maintainability.

Recommended structure:

supply-chain-command-center/

backend/

app/

main.py

config.py

database.py

models.py

schemas.py

api.py

services.py

documents.py

extraction.py

llm_gateway.py

rules_engine.py

traceability.py
recalls.py
security.py
background.py
reporting.py
migrations/
tests/
data/
requirements.txt
Dockerfile
.env.example

frontend/
src/
main.tsx
App.tsx
api.ts
types.ts
components.tsx
layouts.tsx
documents.tsx
validation.tsx
traceability.tsx
recalls.tsx
dashboard.tsx
forms.tsx
styles.css
public/
package.json
Dockerfile

docker-compose.yml
README.md

This is a logical starting point, not a requirement to force every feature into oversized files. Keep a compact repository while preserving separation between API controllers, domain logic, database models, external integrations, and UI components.

Do not create dozens of empty files. Every module must contain working implementation code.

4. Supply-Chain Domain Model

Implement relationships between the following entities:

- Organization
- User
- Role and permission
- Facility
- Supplier
- Customer
- Product
- Raw material
- Bill of Materials

- Supplier certificate
- Purchase Order and Purchase Order Line
- Material Lot
- Goods Receipt and Goods Receipt Line
- Quality Inspection
- Production Order
- Material Consumption
- Finished Product Lot
- Inventory Balance
- Inventory Movement
- Warehouse
- Shipment and Shipment Line
- Delivery
- Return
- Supply-Chain Event
- Lot Genealogy Relationship
- Document
- Document Extraction
- Document Link
- Validation Rule
- Validation Run
- Validation Finding
- Corrective Action
- Recall Case
- Recall Scope Item
- Recall Task
- Customer Notification
- Recall Approval
- Audit Event
- LLM Request and Usage Record

Create suitable foreign keys, indexes, uniqueness constraints, timestamps, lifecycle states, and relationships.

Use database migrations rather than relying solely on automatic table creation.

Store quantities with explicit units. Do not compare or sum incompatible units without a validated conversion.

Support lot splitting, merging, rework, repackaging, scrap, production losses, returns, and retained samples where applicable.

Represent historical supply-chain events using an append-only event ledger. Corrections must create additional audit or correction events instead of silently overwriting historical evidence.

Never expose internal database IDs as the only human-readable identifiers.

Use business identifiers such as:

- PO-2026-000001
- RAWLOT-2026-000001
- GRN-2026-000001
- WO-2026-000001
- FINLOT-2026-000001
- SHP-2026-000001
- DOC-2026-000001
- REC-2026-000001

Generate identifiers safely under concurrent requests.

5. Multi-Document Upload and Management

This is a major product capability, not an optional attachment field.

Build a complete Documents Center supporting multiple document types and multiple files.

Supported file formats

- PDF
- Scanned PDF
- PNG
- JPG and JPEG
- DOCX
- CSV and XLSX for structured operational evidence, using dedicated parsers and validation

Reject unsupported formats with understandable messages.

Configure maximum file size, page count, extraction time, and batch upload limits. Never trust the client-provided file extension or MIME type alone.

Do not accept executable files or unsafe archives as normal documents.

Document categories

Procurement:

- Purchase Order
- Supplier Invoice
- Goods Receipt Note
- Delivery Challan
- Packing List
- Supplier Quotation

Quality:

- Certificate of Analysis
- Material Inspection Report
- Finished Product Inspection Report
- Supplier Quality Certificate
- Calibration Certificate
- Non-Conformance Report

Manufacturing:

- Bill of Materials
- Production Order Report
- Batch Manufacturing Record
- Material Consumption Report
- Rework Report
- Scrap Report

Warehouse and logistics:

- Inventory Report
- Warehouse Transfer Note
- Dispatch Note
- Bill of Lading or equivalent transport document
- Shipment Record
- Proof of Delivery
- Return Receipt

Recall and investigation:

- Customer Complaint
- Recall Notice
- Corrective and Preventive Action Report

- Customer Notification
- Customer Acknowledgement
- Recovery Record
- Disposal Certificate

Also support an OTHER category.

Document types must be extensible through configuration. Do not create a separate hardcoded upload workflow for every type.

Upload behavior

Implement:

- Drag-and-drop upload
- Multi-file upload
- Progress indication
- Processing status
- Retry after recoverable failure
- Duplicate detection using checksum
- Upload metadata
- Document category selection
- Automatic document classification
- Supplier and lot association
- Production order and shipment association
- Recall case association
- Search, filtering, sorting, pagination, and download permissions
- Document processing history
- Document archive/retention state
- Original file preservation

Use private storage. Do not return public file paths or permanent unrestricted storage URLs.

Document processing states

Implement a proper state machine such as:

UPLOADED → QUEUED → EXTRACTING → EXTRACTED → REVIEW_REQUIRED → VERIFIED

Additional terminal or exceptional states:

- FAILED
- REJECTED
- SUPERSEDED

A successful text extraction must not automatically mark a document VERIFIED.

Track the original file checksum, processing timestamps, extractor version, LLM model used when relevant, extraction schema version, and reviewer information.

6. OCR, Text Extraction and Structured Data Extraction

Implement a real pipeline.

1. Validate the file.
2. Save the original file privately.
3. Calculate its checksum.
4. Detect its actual file type.
5. Classify the document.
6. Extract selectable PDF text, DOCX paragraphs and tables, or spreadsheet data.
7. Detect image-only pages.
8. Run OCR on scanned pages when required.

9. Preserve page references and available bounding-box coordinates.
10. Pass relevant extracted text to the LLM for structured extraction.
11. Validate the LLM output against a Pydantic schema.
12. Store extraction results and confidence/provenance information.
13. Flag uncertain fields and contradictions.
14. Present the result for human review.
15. Run deterministic business checks after the necessary fields are accepted.

Do not run OCR unnecessarily on PDFs that already contain reliable text.

If extraction confidence is unavailable or not meaningfully calibrated, use null rather than inventing a confidence percentage.

Every extracted field should preserve provenance wherever feasible:

- Field name
- Extracted value
- Page number
- Source text snippet
- Bounding box when available
- Extraction method
- Confidence when supported
- Review status
- Corrected value, if any

Preserve original extraction results. Human corrections must be recorded as separate reviewed values with actor and timestamp.

Example extraction schema: Invoice

- Document number
- Supplier name
- Supplier tax identifier, if applicable
- Invoice date
- Purchase Order reference
- Currency
- Subtotal
- Tax
- Total
- Line items
- Material codes
- Material descriptions
- Lot or batch references
- Quantities
- Units
- Unit prices
- Line totals
- Payment terms

Example extraction schema: Goods Receipt Note

- GRN number
- Supplier
- Purchase Order reference
- Receipt date
- Warehouse
- Material codes
- Material descriptions

- Supplier lot identifiers
- Internal lot identifiers, if present
- Received quantities
- Units
- Rejected quantities
- Inspection references

Example extraction schema: Certificate of Analysis

- Certificate number
- Issuing organization
- Supplier
- Material
- Lot identifier
- Test name
- Test result
- Unit
- Acceptance limits
- Pass/fail statement if explicitly present
- Test date
- Expiry date, if applicable
- Reviewer or issuing authority, if present

Example extraction schema: Proof of Delivery

- Shipment reference
- Customer
- Delivery location
- Delivery date and time
- Product or lot references
- Delivered quantity
- Unit
- Delivery status
- Recipient acknowledgement
- Exception or damage notes

Create type-specific schemas for other categories.

Never assume that every document contains every field. Distinguish missing fields, unreadable fields, inapplicable fields, and contradictory fields.

7. LLM Gateway Using OpenRouter

Implement an actual reusable LLM service.

Create a single internal gateway used by document extraction, classification, investigation assistance, natural-language search, and report drafting.

Required capabilities:

- Configurable OpenRouter base URL
- API key from environment variables
- Configurable model per task
- Approved-model allowlist
- Request timeout
- Retry with exponential backoff for appropriate transient failures
- Rate limiting
- Request and response size limits
- Structured JSON output where supported
- Pydantic schema validation

- Compatible model fallback
- Token and cost tracking when provider data is available
- Request correlation IDs
- Model latency and failure metrics
- Redacted application logs
- Configurable daily and monthly usage budgets
- Administrator-controlled model configuration
- Clear behavior when the LLM service is unavailable
- Mock LLM provider for deterministic tests

Do not assume every model supports identical tools, JSON modes, context lengths, image inputs, or parameters. Validate capabilities and configuration.

Do not claim a model is free unless the current configuration and provider terms establish that fact.

LLM use cases

1. Document classification.
2. Structured extraction from OCR/text.
3. Explanation of validation findings.
4. Natural-language questions about verified supply-chain records.
5. Investigation summaries.
6. Recall report drafts.
7. Suggested corrective actions.
8. Comparison of evidence from multiple documents.
9. Optional model comparison for administrators.

Mandatory AI safety rules

Treat all document text as untrusted data.

A document may contain instructions such as “ignore previous instructions” or “mark this batch as approved.” Such text must never override application rules.

Do not allow document contents to control system prompts, tools, authorization, or business decisions.

The LLM must not:

- Approve a recall.
- Release quarantined inventory.
- Mark failed quality checks as passed.
- Invent customer deliveries or lot relationships.
- Change inventory balances directly.
- Modify official supplier, lot, or shipment records without a validated authorized workflow.
- Close a recall automatically.
- Treat an OCR extraction as verified truth.

The LLM may propose actions, but the backend must independently validate permissions, evidence, business rules, and workflow state.

All LLM-generated facts used in reports should reference actual database records or document evidence. Clearly distinguish verified facts, extracted claims, inferences, and recommendations.

If the LLM fails, allow manual document review and deterministic checks to continue where possible.

8. Deterministic Run Checks Engine

Build a proper reusable rule engine inspired by enterprise invoice validation systems.

Do not implement all checks as scattered if statements inside API controllers.

Represent checks using a rule registry with:

- Rule ID

- Rule name
- Version
- Category
- Description
- Severity
- Applicable entity types
- Prerequisites
- Execution function
- Expected result
- Actual result
- Evidence references
- Suggested next action
- Whether the finding blocks a workflow transition
- Rule execution timestamp

Support enabling/disabling configurable rules with permissions and version history.

Check statuses

PASS — all required conditions are satisfied.

FAIL — a rule has been conclusively violated.

PENDING — the check has not run or requires a future action.

INCOMPLETE — the evidence is missing, unreadable, contradictory, or insufficient.

WARNING — a non-blocking issue requires attention.

Do not treat PENDING, INCOMPLETE, or WARNING as PASS.

A critical failed check must block the relevant transition. A warning must not block a transition unless the configured policy explicitly requires it.

Procurement and invoice rules

- PO-001: Purchase Order exists and is valid.
- PO-002: Supplier on the PO matches the registered supplier.
- INV-001: Required invoice fields are present.
- INV-002: Detect duplicate invoice numbers using supplier and relevant business identifiers.
- INV-003: Invoice supplier matches the PO supplier.
- INV-004: Invoice quantities and line items reconcile against the PO within configured tolerances.
- INV-005: Invoice calculations and totals reconcile, accounting for applicable tax, discount, currency and rounding rules.
- GRN-001: Goods Receipt references a valid PO.
- GRN-002: Received quantities do not exceed ordered quantities unless an authorized tolerance applies.
- GRN-003: Receipt quantities reconcile against accepted, rejected and outstanding quantities.
- DOC-001: The document’s identifiers match the associated database records.

Quality rules

- QMS-001: Material is eligible for the intended production use.
- QMS-002: Required inspection has been completed.
- QMS-003: Required test results are within configured acceptance limits.
- QMS-004: Expired or invalid certificates are flagged.
- QMS-005: Conflicting document and registered quality statuses require investigation.
- DOC-COA-001: CoA lot identifier matches the registered material lot.
- DOC-COA-002: Required laboratory results are present.

Never release a lot solely because a certificate says PASS.

Manufacturing and genealogy rules

- GEN-001: All material-consumption lot references resolve.
- GEN-002: Production orders reference valid products and BOMs.
- GEN-003: Required inputs and outputs have traceable relationships.
- QTY-001: Material consumption and output reconcile under applicable yield, scrap and unit-conversion rules.
- QTY-002: Actual production does not violate configured tolerances.
- LOT-001: Lot identifiers are unique and required quantities are positive.
- LOT-002: Unit conversions are valid and traceable.
- LOT-003: Rework and split/merge relationships preserve genealogy.

Inventory and logistics rules

- INV-010: Available inventory is calculated from valid movements and reservations.
- INV-011: Quarantined, blocked, recalled, or otherwise ineligible stock cannot be sold or dispatched.
- SHP-001: Shipment lines reference valid lots.
- SHP-002: Shipped quantities do not exceed eligible available inventory after reservations and prior dispatches.
- SHP-003: Required shipment documentation is present.
- SHP-004: Delivery evidence is consistent with the shipment.
- RET-001: Returned quantities are linked to a valid shipment or authorized exception.

Recall rules

- RCL-001: Recall scope has been simulated before approval.
- RCL-002: Approval requires an authorized actor and justification.
- RCL-003: Activation blocks affected stock and prevents eligible new dispatches.
- RCL-004: All traceable affected customers and shipments are included in the scope or explicitly resolved as exceptions.
- RCL-005: Required notifications are recorded.
- RCL-006: Recovery quantities reconcile with dispatched and affected quantities, with documented exceptions.
- RCL-007: A recall cannot close while required tasks, critical findings or mandatory acknowledgements remain unresolved under the configured policy.
- RCL-008: Every closure decision has evidence and an audit trail.

Audit and AI rules

- AUD-001: Critical state transitions record actor, timestamp, reason, and previous/new states.
- AUD-002: Corrections to reviewed evidence preserve history.
- AI-001: LLM output cannot directly mutate authoritative domain state.
- AI-002: Generated claims must reference valid evidence where required.
- AI-003: Model failures and validation failures are logged safely.

Execute checks on demand and automatically at relevant workflow transitions.

Every run must be persisted. Provide an API to retrieve runs and findings later.

9. Document Validation User Experience

Create a dedicated Documents Center with:

Documents list

Columns:

- Document ID
- Filename
- Document type
- Supplier
- Associated lot
- Related PO/GRN/shipment/recall
- Upload date

- Processing state
- Review state
- Latest validation status
- Uploaded by
- Actions

Document detail screen

Use a two-panel layout.

Left panel:

- PDF preview or image viewer
- Page navigation
- Zoom
- Download permission
- Highlighted source regions where available

Right panel:

- Extracted structured fields
- Confidence/provenance
- Missing required fields
- Field-level warnings
- Database-matched values
- Differences between document values and registered records
- Editable review fields
- Save corrections
- Accept or reject extraction
- Link to business records
- Run Checks button
- Processing history

Show clearly when a value was machine-extracted versus human-reviewed.

Do not display fabricated confidence percentages.

Validation Center

Create a separate screen containing:

- Total checks
- Pass count
- Fail count
- Pending count
- Incomplete count
- Warning count
- Critical failures
- Filters by rule, severity, entity, supplier, lot, document and date
- Finding details
- Expected versus actual values
- Evidence links
- Recommended next action
- Rerun action where permitted
- Resolution history

A finding must link back to the relevant source document and business entity.

10. Supply-Chain Traceability

Implement a working interactive genealogy graph.

Users should be able to search for a material lot, finished-product lot, shipment, production order, or recall case.

Support upstream tracing:

- Finished lot
- Production order
- Consumed material lots
- Suppliers
- Incoming inspections
- Relevant source documents

Support downstream tracing:

- Raw-material lot
- Production orders consuming it
- Finished lots produced
- Warehouse locations
- Shipments
- Customers and deliveries
- Returns and recall cases

Include filters, node details, event history, relevant validation findings and document links.

Provide an understandable table/list alternative to the graph.

Detect missing references, cycles where invalid, incomplete genealogy, inconsistent quantities and unresolved relationships.

Do not infer missing relationships from LLM guesses.

11. Recall Management Workflow

Implement a complete, auditable workflow.

1. Create an investigation from a complaint, failed inspection, supplier alert or other evidence.
2. Link the originating evidence and affected entity.
3. Run upstream and downstream traceability.
4. Simulate the proposed recall scope.
5. Identify affected raw lots, finished lots, inventory, shipments and customers.
6. Show quantities and unresolved genealogy gaps.
7. Run critical recall checks.
8. Require authorized human approval with justification.
9. Activate the recall.
10. Quarantine or block affected inventory using validated domain services.
11. Prevent eligible future shipments.
12. Generate customer notification drafts.
13. Record notifications, acknowledgements, returns and recovered quantities.
14. Track corrective actions and exceptions.
15. Reconcile quantities and unresolved findings.
16. Require authorized closure after configured mandatory conditions are met.

Use database transactions and concurrency protection for critical state transitions.

Do not let the frontend bypass workflow validation by directly editing a status field.

If notifications are simulated in development, label them clearly as simulated. Do not claim real emails or messages were sent unless a configured delivery integration confirms success.

12. Frontend Application Screens

Build these functional screens:

1. Login and access-denied pages.
2. Executive Dashboard.
3. Suppliers.
4. Products and Materials.
5. Purchase Orders.
6. Goods Receipts.
7. Material Lots.
8. Quality Inspections.
9. Production Orders.
10. Inventory and Warehouses.
11. Shipments and Deliveries.
12. Documents Center.
13. Document Review and Extraction.
14. Validation Center.
15. Traceability Explorer.
16. Recall Cases.
17. Recall Investigation and Scope.
18. Corrective Actions.
19. Reports and Exports.
20. Audit Trail.
21. LLM Usage and Model Settings for authorized administrators.
22. Application Settings.

Build functional navigation, search, filters, loading states, empty states, error states, confirmation dialogs, pagination and toast messages.

Use consistent spacing, typography, accessible contrast, keyboard support, responsive tables, clear status indicators and professional enterprise styling.

Avoid a generic AI landing page aesthetic. This is an operational application for supply-chain and risk professionals.

The frontend must use real backend endpoints. Do not create fake hardcoded dashboard metrics when database-backed data is available.

For development/demo mode, display a clear demo-data indicator.

13. Backend API

Use versioned routes under /api/v1.

Implement at minimum:

Documents

- POST /documents
- POST /documents/batch
- GET /documents
- GET /documents/{document_id}
- GET /documents/{document_id}/content
- POST /documents/{document_id}/process
- GET /documents/{document_id}/processing-status
- GET /documents/{document_id}/extraction
- PATCH /documents/{document_id}/review
- POST /documents/{document_id}/links
- GET /documents/{document_id}/links

- POST /documents/{document_id}/run-checks

Validation

- GET /checks/rules
- POST /checks/runs
- GET /checks/runs/{run_id}
- GET /checks/findings
- GET /checks/findings/{finding_id}
- POST /checks/findings/{finding_id}/resolve

Supply chain

- CRUD endpoints for suppliers, products, materials, purchase orders, receipts, lots, production orders, inventory, shipments and deliveries.
- GET /lots/{lot_id}/events
- GET /genealogy/{entity_id}
- GET /inventory
- POST /imports
- GET /imports/{import_id}

Recall

- POST /recalls
- GET /recalls
- GET /recalls/{recall_id}
- POST /recalls/{recall_id}/simulate
- POST /recalls/{recall_id}/approve
- POST /recalls/{recall_id}/activate
- POST /recalls/{recall_id}/notifications
- POST /recalls/{recall_id}/reconcile
- POST /recalls/{recall_id}/close

AI

- POST /ai/document-classification
- POST /ai/extract
- POST /ai/ask
- POST /ai/investigation-summary
- POST /ai/recall-report-draft
- GET /ai/usage

Validate all inputs and permissions. Return consistent error structures and appropriate HTTP status codes. Add request IDs and pagination to relevant endpoints.

Use OpenAPI documentation generated by FastAPI.

14. Security and Data Integrity

Implement:

- Authentication integration or a documented secure local development authentication mode
- Role-based access control
- Organization/facility-level access scoping where applicable
- Permission checks on document viewing, downloading, reviewing, approving and closing
- File type and size validation
- Private file storage
- Secret management via environment variables
- Rate limits on expensive AI endpoints
- Protection against path traversal
- Parameterized database access

- Safe logging and error handling
- Audit logging for sensitive actions
- CORS configured from environment settings
- Safe handling of uploaded prompt-injection content
- Concurrency control for inventory and recall transitions
- Idempotency for important write operations
- Configurable data retention
- Database constraints and transactions

Never store plaintext passwords. Never commit real credentials. Do not return stack traces or secrets to normal API clients.

Clearly document any security features that remain development-only.

15. Seed Dataset and Demonstration

Generate reproducible demo data using a fixed random seed.

Target dataset:

- 500 suppliers
- 10,000 raw material lots
- 5,000 production orders
- 8,000 finished-product lots
- 20,000 deliveries

Use deterministic generation, valid relationships, realistic quantities and dates, and a consistent unit system.

Include several deliberate data-quality anomalies in a separate ground-truth file:

1. A finished lot references a missing input lot.
2. Delivered quantity exceeds produced quantity.
3. A recall simulation misses a downstream customer.
4. Recalled inventory remains available for sale.

Do not expose ground-truth labels to operational validation logic. Use them only in tests and evaluation.

Provide smaller seed data for fast local development, along with an option to generate the full dataset.

Generate CSV/JSON seed files or a repeatable database seeding script, a data dictionary and a data-validation report.

16. Automated Testing

Build real tests for the backend, frontend integration and major workflows.

Backend:

- Unit tests for document parsers and schemas
- Unit tests for each critical validation rule
- API integration tests
- Database constraint tests
- Document upload security tests
- LLM gateway tests using mocked responses
- LLM timeout, malformed JSON and provider failure tests
- Recall lifecycle and authorization tests
- Inventory concurrency tests
- Idempotency tests
- Regression tests for all planted anomalies

Frontend:

- Component tests
- Form validation tests

- Document upload and processing-status tests
- Review and correction tests
- Validation findings tests
- Traceability rendering tests
- Recall approval and closure workflow tests

End-to-end:

1. Upload a PO.
2. Extract its fields.
3. Review the extraction.
4. Link it to a supplier.
5. Upload an invoice and compare it with the PO.
6. Upload a GRN and validate quantities.
7. Inspect a CoA and quality status.
8. Run traceability on a finished lot.
9. Create a recall simulation.
10. Verify that critical failures block unauthorized activation or closure.

The test suite must work without a paid LLM account or live external model requests.

17. Observability and Reports

Record:

- Request IDs
- Document processing durations
- Extraction failures
- LLM latency and failures
- Token usage and cost when available
- Validation run counts and outcomes
- Recall workflow events
- Background task failures
- Retry counts

Provide downloadable CSV reports and an appropriate PDF report for recall investigations.

Recall reports must distinguish verified database facts, reviewed document evidence, unresolved findings, LLM-generated recommendations and human decisions.

Do not silently omit missing data or failed checks from reports.

18. Engineering Quality Requirements

The project must contain substantive implementation, not empty placeholders.

Do not use:

- pass in place of core functionality
- TODO for required features
- Fake API responses
- Mock-only document extraction presented as real OCR
- Random validation outcomes
- Hardcoded dashboard metrics presented as live data
- A frontend that bypasses the backend
- In-memory-only storage for important production records
- LLM-generated values treated as verified business facts

Use mocks only for tests and explicitly labeled demo modes.

Aim for more than 3,000 meaningful source lines across the complete application if needed to implement the requested scope properly. Do not inflate line counts with repeated comments, filler, generated junk or unnecessary abstractions. Correctness and completeness matter more than a line-count target.

If the entire implementation cannot be completed in one pass, build it in ordered, runnable phases and report precisely which features are complete, which are partial, and which remain. Never claim an unimplemented feature is finished.

19. Build Order

Implement in this order:

Phase 1: Repository setup, configuration, database models, migrations, seed data, authentication foundation and shared UI.

Phase 2: Supplier, material, lot, production, inventory and shipment APIs.

Phase 3: Document storage, upload UI, file validation, OCR, extraction schemas and review workflow.

Phase 4: OpenRouter gateway, task-based model routing, structured extraction, usage controls and graceful fallback.

Phase 5: Deterministic Run Checks engine, persisted findings, validation UI and evidence links.

Phase 6: Lot genealogy, traceability graph and quantity reconciliation.

Phase 7: Recall simulation, approval, activation, notifications, recovery, closure and audit history.

Phase 8: Reports, dashboard analytics, security hardening, comprehensive tests and Docker deployment.

After each phase, run the relevant tests and resolve failures before proceeding.

20. Definition of Done

The application is ready for handover only when:

- The frontend starts successfully.
- The backend starts successfully.
- PostgreSQL migrations work from a clean database.
- Demo data can be seeded repeatably.
- Multiple document types can actually be uploaded.
- PDF text extraction and scanned-document OCR work.
- The LLM can extract structured fields through configured OpenRouter models.
- Users can review and correct extracted values.
- Documents can be linked to business entities.
- Run Checks persist results and provide evidence-based findings.
- Critical validation failures block the relevant business transitions.
- Lot genealogy traces upstream and downstream.
- Recall simulation identifies affected stock and customers based on recorded relationships.
- Recall approvals and closure are permission-controlled and auditable.
- AI failures do not corrupt authoritative data.
- Automated tests cover critical business rules and planted anomalies.
- Docker and environment configuration are documented.
- The README includes installation, configuration, migrations, seeding, test execution, API documentation, OCR dependencies and troubleshooting.

Begin by inspecting the current repository and preserving existing working functionality. Then produce an implementation plan, identify any missing prerequisites, and start building the application. Do not stop after presenting the plan. Implement, run tests, fix errors, and report verified results.

Prompt 10

Title	Make “Trace a material or finished lot” work from uploaded evidence
Tool	Not recorded in the supplied material (coding-assistant style prompt)
Source	Pasted text supplied in a follow-up message (“this prompt also used so add it”)
Note	Added after the first version of this document. The supplied text does not say which tool received it, when it was sent, or where it falls in the sequence. It mentions a configured LLM (Gemini or similar) and GEMINI_API_KEY, but that does not establish which tool was used. Confirm the tool before submission.

PROMPT TEXT

Project structure:

- Frontend: traceiq-frontend — React, TypeScript, Vite
- Backend: traceiq-backend — FastAPI, SQLAlchemy, Pydantic
- The existing backend owns all authoritative traceability, quality, quantity, and recall decisions.

GOAL

Make “Trace a material or finished lot” work from uploaded evidence. An operator should be able to upload supply-chain documents, extract candidate data with the configured LLM, review the extraction, link verified information to real records, and trace the resulting lot relationships.

START BY AUDITING THE CURRENT FLOW

Inspect the existing implementation before changing it, especially:

- Frontend: src/screens/TraceabilityScreen.tsx, src/screens/DocumentsScreen.tsx, and src/services/api/
- Backend: app/controllers.py, app/documents.py, app/services.py, app/models.py, app/ai.py, and app/main.py
- Existing upload, extraction, review, document-link, lot, traceability, quality, and audit endpoints

Identify and report which parts are currently demo/static, including:

- Any hardcoded initial lot IDs such as RAW-1042
- Seeded demo lots and relationships
- Sample data that may be presented as live records
- Document extraction that uses deterministic readers only
- Extracted document fields that aren’t connected to lots or the trace graph

Then implement the feature end to end. Do not stop after producing a plan.

REQUIRED USER FLOW

1. The trace screen must not silently open on a hardcoded demo lot. Provide a clear empty state where users can search for an existing lot or upload evidence.

2. Add an “Upload evidence” flow to the trace screen, reusing the existing document upload API and document processing behavior where practical.

3. Support the document types already supported by TraceIQ. For PDFs, extract text and use the configured LLM to identify candidate supply-chain fields. Handle scanned PDFs with OCR if the current stack supports it; otherwise show a clear limitation.

4. Extract candidate values such as:

- Material or finished lot IDs and names
- Supplier IDs or names
- Production order IDs
- Input and output lot relationships
- Quantities and units
- Shipment, inventory, receipt, and customer references when present

5. Return structured, schema-validated extraction results. For every candidate field, retain its source page or location, source snippet, method, and confidence. Use null or “not found” for missing values rather than guessing.

6. Show the operator a review step. Let them correct, accept, or reject candidate fields and relationships. Clearly distinguish LLM-extracted candidates from verified business records.

7. Only create or link authoritative lot relationships after operator confirmation and backend validation. Use the existing document-link and audit facilities where possible.

8. After confirmation, refresh the trace graph from backend records and show the uploaded documents as supporting evidence for the relevant entities.

9. If a document does not contain enough information, show what is missing and let the operator continue with manual entry or upload more evidence.

10. Keep live records and demo fixtures visibly separated. Never mix seeded demo data into a live trace without labeling it.

LLM AND DATA SAFETY

- Use the existing configured LLM integration where possible. If Gemini is the configured provider, read its credential from backend environment settings such as GEMINI_API_KEY.
- Never put an LLM API key in frontend code, commit it, or copy a pasted key into source files.
- Treat LLM output as untrusted candidate data. Validate types, identifiers, quantities, units, and relationships on the backend.
- Keep deterministic quality gates, quantity calculations, recall scope, and approval decisions in backend business logic. The LLM must not approve lots, quality checks, or recalls.
- Preserve source evidence and record an audit event for confirmed changes.
- Add sensible timeouts and clear user-facing errors for unsupported, unreadable, or oversized files.

IMPLEMENTATION EXPECTATIONS

- Extend the current FastAPI controllers/services/schemas and database models only where needed; include migrations if the project uses them.
- Reuse existing frontend API client, loading/error/empty states, notifications, and teal-and-white design.
- Make the trace screen responsive and easy to follow: upload/search → extraction review → confirmed records → trace graph.
- Keep demo mode available only when explicitly selected and label its data as illustrative.
- Do not fabricate supplier, lot, quantity, or genealogy values.

ACCEPTANCE CRITERIA

- Uploading a supported PDF creates a document record and displays processing status.
- Processing returns structured candidate fields with evidence and confidence, or a clear explanation when extraction fails.
- The operator can edit and confirm extracted data; rejected or unconfirmed candidates do not become authoritative trace records.
- Confirmed links appear in the live trace graph after refresh and retain a link to their source document.
- Searching an unknown lot shows a useful empty state instead of silently rendering RAW-1042.
- Existing manual lot intake, production, document review, quality checks, and traceability behavior continue working.
- Add or update backend and frontend tests for upload, extraction, review, linking, and trace refresh.
- Run the project’s lint/build and backend test commands, then report changed files, results, environment variables, and run commands.

Prompt 11

Title	Request to compile this prompt-history PDF (and follow-up)
Tool	Claude (Anthropic)
Source	This conversation
Note	The request that produced this document. Included for completeness because it was also submitted to an AI tool.

PROMPT TEXT

i have mentioned all the prompts used to develop an project so i want prompt history is required in one pdf folder

Include all prompts submitted to every AI tool you used. AI

responses are optional. If a tool cannot export prompts, keep a

reasonable record and disclose the limitation.

[Attached files: traceiq-_-supply-chain-traceability-_-ai-recall-command-center.zip; AI_Prompt_History_Supply_Chain_Traceability.docx; TraceIQ_Product_Requirements_Document__1_.docx. The message also carried the text of the earlier prompts reproduced in this document.]

[Follow-up message in the same conversation, after the PDF was produced:]

this prompt also used so add it

[Attached text: the “Make ‘Trace a material or finished lot’ work from uploaded evidence” prompt, now recorded as Prompt 10.]

5. Appendix — outcome notes from the Gemini transcript (optional)

AI responses are optional. The source transcript recorded these short outcome summaries; they are included only for context.

After prompt	Run time	Outcome recorded in the transcript
1	11m 11s	19 files edited; build reported successful. Dark navy enterprise redesign across six pages.
2	9m 15s	16 files edited; build reported successful. Redesign of an uploaded earlier version, with a scenario walkthrough bar.
3	—	Run cancelled; restored to previous checkpoint. No changes kept.
4	7m 19s	14 files edited; build reported successful. Light-theme redesign of all six pages.